Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 8

Team No.: 11

Project Title: Linux Inter-Process Communication Using Pipes and FIFO

Team Members

Roll Number	Student Name	Signature
2520030340	Srikruthi	
2520030007	Sathwika	
2520030511	Akhila	
2520030117	Vaishali	

1. Abstract

Inter-Process Communication (IPC) is an essential concept in operating systems that allows processes to exchange information and coordinate tasks. In Linux, two common IPC mechanisms are pipes and FIFOs (named pipes). Pipes provide a simple communication channel between related processes, such as a parent and child created using fork(). They enable one process to write data while another reads it, ensuring smooth data transfer without using external files. FIFOs extend this idea by creating a special file in the filesystem, allowing unrelated processes to communicate through a shared channel.

This project demonstrates IPC using both pipes and FIFOs. By implementing small programs that send and receive text messages, the system highlights how processes can cooperate and share resources effectively. The project uses system calls like pipe(), mkfifo(), read(), and write() to establish communication channels and transfer data. Pipes illustrate basic parent-child communication, while FIFOs show how independent processes can interact through a named pipe.

The work emphasizes the importance of IPC in concurrent programming, where multiple processes must work together to achieve a common goal. It provides practical experience with Linux system calls and process management, bridging the gap between simple process creation and advanced synchronization techniques.

2. Problem Statement

In operating systems, processes often run at the same time and need to share information. Without a way to communicate, these processes remain isolated, making cooperation and synchronization difficult. This limits efficiency and the ability to perform tasks that require data exchange.

Linux provides Inter-Process Communication (IPC) mechanisms such as pipes and FIFOs (named pipes) to solve this problem. Pipes allow related processes, like a parent and child, to send and receive data directly. FIFOs extend this capability to unrelated processes by using a special file in the filesystem.

The problem addressed in this project is the lack of understanding and demonstration of how processes can communicate effectively in Linux. By implementing IPC using pipes and FIFOs, the project shows how processes can share data, coordinate actions, and work together, which is a core concept in operating systems and system programming.

3. Objectives

1. To demonstrate how processes in Linux can communicate and share data using pipes and FIFOs (named pipes)

2. To implement simple programs that use system calls (pipe(), mkfifo(), read(), write()) for message transfer between processes.

3. To highlight the difference between communication in related processes (via pipes) and unrelated processes (via FIFOs).

4. To provide practical understanding of IPC concepts in operating systems, showing how process cooperation improves efficiency and synchronization

4. Proposed Methodology

The project will be designed and implemented in a Linux environment using the C programming language and standard system calls. The methodology involves creating small programs that demonstrate communication between processes using pipes and FIFOs (named pipes).

1. Process Creation: A parent process will be created using fork(), which generates a child process. This setup allows testing of communication between related processes.

2. Pipe Implementation: Using the pipe() system call, a unidirectional communication channel will be established. The parent process will write data into the pipe, and the child process will read it, demonstrating simple message passing.

3. FIFO Implementation: A named pipe will be created using mkfifo(). Two independent processes will be programmed to communicate through this FIFO file. One process will write data, while the other will read it, showing communication between unrelated processes.

4. Data Transfer and Synchronization: System calls such as read() and write() will be used to send and receive messages. The flow will ensure proper synchronization so that data is transferred reliably between processes.

5. Testing and Validation: Sample messages will be exchanged to verify functionality. The results will demonstrate how IPC mechanisms enable cooperation and efficient communication in Linux.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Fork( )	To create a child process for demonstrating communication between parent and child.
Pipe( )	To establish a unidirectional communication channel between related processes.
mkfife( )	To create a named pipe (FIFO) for communication between unrelated processes.

read( )	To read data sent through a pipe or FIFO from another process
write( )	To send data through a pipe or FIFO to another process.
close( )	To close file descriptors after communication is complete, ensuring resource management.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030340	Srikruthi	Handled testing, debugging, and validation of data transfer between processes
2520030007	Sathwika	Designed and implemented pipe communication between parent and child processes
2520030511	Akhila	Prepared documentation, report formatting, and presentation slides.
2520030117	Vaishali	Developed FIFO (named pipe) communication between unrelated processes

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	To provide a stable environment for compiling and executing system-level programs using IPC mechanisms
C / C++	To implement IPC using system calls like pipe(), mkfifo(), read(), and write() for process communication.
GCC Complier	To compile and build the C programs for testing and execution.
Terminal / Shell	To run and monitor processes, execute commands, and observe communication results.

8. Expected Outcome

The project is expected to demonstrate successful communication between processes using pipes and FIFOs in a Linux environment. It will show how related and unrelated processes can exchange data efficiently through system calls. The outcome includes a working program that transfers messages between processes, validating the concept of Inter-Process Communication (IPC). This will help in understanding process synchronization, resource sharing, and the practical implementation of OS-level communication mechanisms.

9. GitHub Repository

GitHub Repository Link: https://github.com/Sathwika-019/Prime_OSSP

Repository Created: Yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Muniraj Sir

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____